

Dokumentacja projektu Neo4J Movies Visual Studio

część dotycząca elementów związanych z bazą danych

Węzły (nodes)

User – węzeł odpowiedzialny za przechowywanie danych o użytkownikach. Posiada następujące właściwości:

- **email**,
- **firstName**,
- **lastName**,
- **password** (hash hasła wygenerowany za pomocą biblioteki **Bcrypt**),
- **role** (**admin/user**, ale w obecnej wersji dostępna jest tylko rola **user**),
- **username** (unikalny pseudonim użytkownika, który wykorzystywany jest przy logowaniu).

Movie – węzeł odpowiedzialny za przechowywanie danych o filmach. Posiada następujące właściwości:

- **genre** (lista gatunków w postaci napisu),
- **poster** (link do plakatu filmu, obecna wersja posiada lokalny folder z plakatami, ponieważ serwer dostawcy nie nadążał z przesyłaniem zdjęć w niektórych przypadkach),
- **released** (rok wydania),
- **title**.

Genre – węzeł ułatwiający przechodzenie po grafie. Posiada jedną właściwość: **genreTitle**.

Testowana baza posiadała 41 węzłów **Movie**, 13 użytkowników oraz 14 gatunków. Użytkownicy zostali utworzeni ręcznie przez interfejs graficzny. Natomiast filmy i gatunki zostały dodane przez następującą kwerendę:

```
CREATE
(:Movie {title: 'Iron Man', genre: 'Action, Sci-Fi', released: 2008, poster:
'https://en.wikipedia.org/wiki/Special:FilePath/Iron_Man_(2008_film)_poster.jpg'}),
(:Movie {title: 'The Incredible Hulk', genre: 'Action, Sci-Fi', released: 2008,
poster:
'https://en.wikipedia.org/wiki/Special:FilePath/The_Incredible_Hulk_(film)_poster.jpg'
}),
(:Movie {title: 'Iron Man 2', genre: 'Action, Sci-Fi', released: 2010, poster:
'https://en.wikipedia.org/wiki/Special:FilePath/Iron_Man_2_poster.jpg'}),
(:Movie {title: 'Thor', genre: 'Action, Fantasy', released: 2011, poster:
'https://en.wikipedia.org/wiki/Special:FilePath/Thor_(film)_poster.jpg'}),
(:Movie {title: 'Captain America The First Avenger', genre: 'Action, Adventure',
released: 2011, poster:
'https://en.wikipedia.org/wiki/Special:FilePath/Captain_America_The_First_Avenger_poster.jpg'}),
```

...

```
MATCH (m:Movie)
WITH m, split(m.genre, ',') AS genres
UNWIND genres AS genreName
MERGE (g:Genre {genreTitle: trim(genreName)})
MERGE (m)-[:IN_GENRE]->(g);
```

Relacje (relations)

LIKED – reprezentuje polubienie filmu przez użytkownika (User → Movie).

OBSERVES – pozwala użytkownikowi na śledzenie aktywności innych osób w systemie (User → User).

IN_GENRE – ułatwia proces przechodzenia po grafie podczas rekomendacji po gatunkach (Movie → Genre).

Funkcje C# operujące na bazie danych (Neo4jDriver.cs)

Klasa posiada statyczny obiekt **Instance** utworzony na podstawie wzorca **Singleton**, który używany jest do komunikacji z bazą.

`public async Task<bool> UserExists(string username)` – funkcja sprawdza czy użytkownik o podanym pseudonimie istnieje w bazie. Zwraca true, jeżeli pseudonim jest zajęty, w przeciwnym wypadku zwraca false. Zapobiega tworzeniu duplikatów.

```
var query = @"
MATCH (u:User {username: $username})
RETURN u
LIMIT 1";
```

`public async Task CreateUser(string firstName, string lastName, string email, string password, string username)` – funkcja odpowiedzialna za utworzenie użytkownika.

```
var query = @"
CREATE (u:User {
    username: $username,
    firstName: $firstName,
    lastName: $lastName,
    email: $email,
    password: $password,
    role: 'user'
})";
```

`public async Task <bool> VerifyUser(string username, string password)` – funkcja wykorzystywana do weryfikacji użytkownika. Zwraca true, jeżeli weryfikacja się powiedzie, w przeciwnym przypadku zwraca false.

```
var query = @"
MATCH (u:User { username: $username })
RETURN u.password";
```

`public async Task<User> GetUserData(string username)` – funkcja zwracająca wszystkie informacje o poszukiwanym użytkowniku. Zwraca null w przypadku braku użytkownika o podanym pseudonimie.

```
var query = @"
    MATCH (u:User { username: $username })
    RETURN u.firstName AS firstName,
           u.lastName AS lastName,
           u.email AS email,
           u.role AS role";
```

`public async Task<List<Movie>> GetMovies()` – funkcja zwracająca wszystkie informacje o filmach.

```
var query = @"MATCH (m:Movie)
    RETURN m.title AS title,
           m.genre AS genre,
           m.released AS released,
           m.poster AS poster";
```

`public async Task<bool> CreateLikeMovieRelation(string username, string movieTitle)` – funkcja odpowiedzialna za utworzenie relacji polubienia filmu. Zwraca true w przypadku powodzenia, w przeciwnym wypadku zwraca false.

```
var query = @"
    MATCH (u:User { username: $username }),
          (m:Movie { title: $movieTitle })
    MERGE (u)-[:LIKED]->(m)
    RETURN u, m";
```

`public async Task<bool> CreateObservesRelation(string currentUserUsername, string targetUsername)` – funkcja tworząca relacje obserwowania wybranego użytkownika. Zwraca true w przypadku powodzenia lub false przy niepoprawnych danych.

```
var query = @"
    MATCH (u:User { username: $currentUserUsername }),
          (t:User { username: $targetUsername })
    MERGE (u)-[:OBSERVES]->(t)
    RETURN u, t";
```

`public async Task<HashSet<string>> GetLikedMovieTitles(string username)` – funkcja odpowiedzialna za zwrócenie listy polubionych filmów, wspomagająca ustawienie właściwości interfejsu graficznego.

```
var query = @"
    MATCH (u:User {username: $username})-[:LIKED]->(m:Movie)
    RETURN m.title AS Title";
```

`public async Task<bool> RemoveLikeMovieRelation(string username, string movieTitle)` – funkcja odpowiedzialna za usunięcie relacji polubienia filmu. Zwraca true w przypadku powodzenia, w przeciwnym przypadku false.

```
var query = @"MATCH (u:User { username: $username })-[r:LIKED]->(m:Movie
{ title: $movieTitle })
    DELETE r
    RETURN count(r) AS deletedCount";
```

`public async Task<List<object>> GetUsers(string currentUser)` – funkcja zwracająca pseudonimy wszystkich użytkowników z wykluczeniem zalogowanego użytkownika.

```
string query = @"
    MATCH (u:User)
    WHERE u.username <> $currentUser
    RETURN u.username AS username";
```

`public async Task<HashSet<string>> GetFollowedUsers(string username)` – funkcja zwracająca obserwowanych użytkowników przez podanego użytkownika.

```
var query = @"
    MATCH (u:User {username: $username})-[:OBSERVES]->(f:User)
    RETURN f.username AS FollowedUsername";
```

`public async Task<bool> RemoveObservesRelation(string followerUsername, string followedUsername)` – funkcja usuwająca relacje obserwowania danego użytkownika. Zwraca true w przypadku powodzenia, w przeciwnym wypadku zwraca false.

```
var query = @"
    MATCH (follower:User { username: $followerUsername })-[r:OBSERVES]->(followed:User
{ username: $followedUsername })
    DELETE r
    RETURN count(r) AS deletedCount";
```

`public async Task<List<Movie>> RecommendMoviesForUser(string username, int limit = 10)` – funkcja zwracająca listę rekomendowanych filmów na podstawie polubień obserwowanych użytkowników.

```
var query = @"
    MATCH (u:User {username: $username})-[:OBSERVES]->(f:User)-[:LIKED]>(m:Movie)
    WHERE NOT (u)-[:LIKED]->(m)
    RETURN m.title AS title, m.genre AS genre, m.released AS released, m.poster AS
poster, count(*) AS score
    ORDER BY score DESC
    LIMIT $limit";
```

`public async Task<List<Movie>> RecommendMoviesByGenre(string username, int limit = 10)` – funkcja zwracająca listę rekomendowanych filmów na podstawie gatunków filmów polubionych przez użytkownika.

```
var query = @"
    MATCH (u:User {username: $username})-[:LIKED]->(:Movie)-[:IN_GENRE]->(g:Genre)
    MATCH (g)-[:IN_GENRE]-(m:Movie)
    WHERE NOT (u)-[:LIKED]->(m)
    RETURN m.title AS title,
           m.genre AS genre,
           m.released AS released,
           m.poster AS poster,
           count(*) AS score
    ORDER BY score DESC
    LIMIT $limit";
```